



UNIVERSIDAD PRIVADA DE TACNA

FACULTAD DE INGENIERIA

Escuela Profesional de Ingeniería de Sistemas

INFORME DE LABORATORIO
“Videojuego tipo Snake”

Curso: Programación II

Docente: Ing. Breno Luque Zuñiga

Tarqui Ramos, Sebastián Emir Andree (2022075482)

Tacna – Perú
2026



INDICE

I. INFORMACIÓN GENERAL	3
- Objetivos:	3
- Equipos, materiales, programas y recursos utilizados:	3
II. MARCO TEORICO	3
III. PROCEDIMIENTO	4
IV. ANALISIS E INTERPRETACION DE RESULTADOS	10
V. CUESTIONARIO	11
CONCLUSIONES	11



INFORME DE LABORATORIO

Videojuego tipo Snake

I. INFORMACIÓN GENERAL

- **Objetivos:**
 - Diseñar e implementar un videojuego tipo Snake utilizando C# y Windows Forms.
 - Aplicar los principios de Programación Orientada a Objetos: clases, encapsulamiento, herencia, colecciones y eventos.
 - Separar responsabilidades entre clases (Serpiente, Alimento, Juego, Form1).
 - Implementar detección de colisiones, control de puntaje y lógica de reinicio.
- **Equipos, materiales, programas y recursos utilizados:**
 - Computadora con Windows 10/11
 - Visual Studio 2022
 - Lenguaje de programación: C# (.NET Framework)
 - Tipo de proyecto: Windows Forms Application
 - Librerías: System.Drawing (GDI+), System.Windows.Forms, System.Collections.Generic.

II. MARCO TEORICO

El videojuego Snake es un clásico donde el jugador controla una serpiente que se desplaza por un tablero. Al consumir alimentos, la serpiente crece y el puntaje aumenta. El juego termina si la serpiente choca contra los bordes o contra su propio cuerpo.

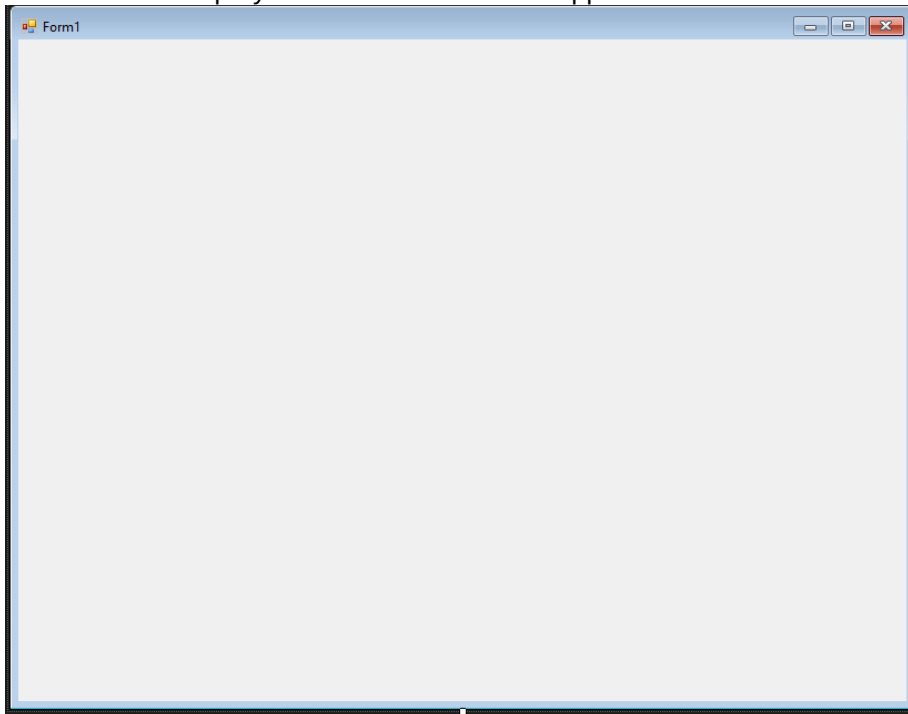
Para este proyecto se aplican los siguientes conceptos de POO:

-Clases y objetos: cada entidad del juego (Serpiente, Alimento, Juego) es una clase independiente con sus atributos y métodos.

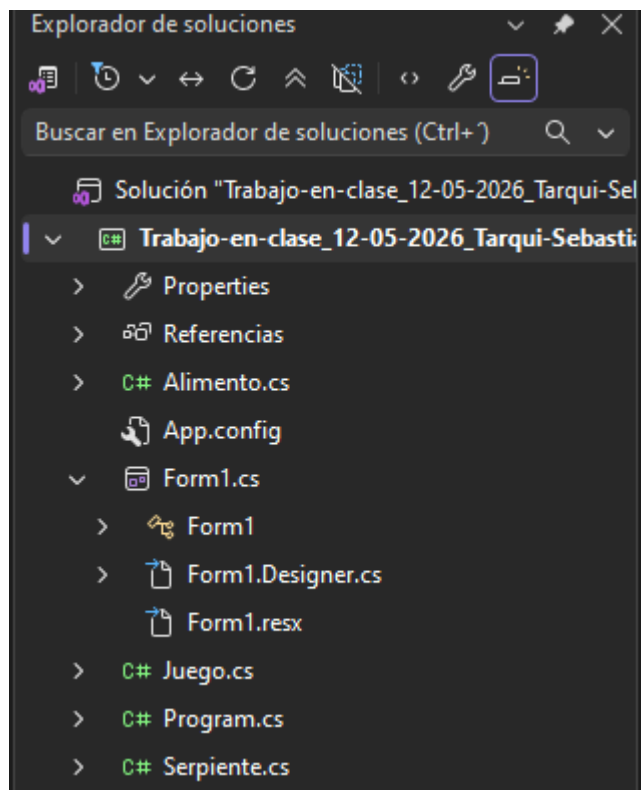
- Encapsulamiento: los atributos internos son privados y se acceden mediante propiedades públicas.
- Colecciones: la serpiente utiliza List<Point> para gestionar sus segmentos dinámicamente.
- Enumeración: el tipo Dirección define las 4 direcciones posibles de forma legible y segura.
- Eventos y temporizadores: el Timer dispara el loop del juego cada 120 ms; KeyDown captura el teclado.
- GDI+: la clase Graphics de System.Drawing dibuja el tablero, la serpiente y el alimento sobre un Panel.
- Separación de responsabilidades: Form1 solo maneja la UI y delega la lógica al objeto Juego.

III. PROCEDIMIENTO

1. Se creo un proyecto Windows Forms App en Visual Studio 2022 con el nombre Form1.



2. Se agregaron las clases Serpiente.cs, Alimento.cs y Juego.cs al proyecto.





```
10 public class Alimento
11 {
12     private Point posicion;
13     private Random random;
14
15     3 referencias
16     public Point Posicion => posicion;
17
18     1 referencia
19     public Alimento()
20     {
21         random = new Random();
22     }
23
24     3 referencias
25     public void GenerarNuevaPosicion(int columnasTablero, int filasTablero, List<Point> segmentosSerpiente)
26     {
27         do
28         {
29             posicion = new Point(random.Next(0, columnasTablero), random.Next(0, filasTablero));
30         } while (segmentosSerpiente.Contains(posicion));
31     }
32 }
```

```
10 23 referencias
11 public enum Direccion { Arriba, Abajo, Izquierda, Derecha }
12 4 referencias
13 public class Serpiente
14 {
15     private List<Point> segmentos;
16     9 referencias
17     public Direccion DireccionActual { get; private set; }
18     0 referencias
19     public int Tamano => segmentos.Count;
20
21     1 referencia
22     public Serpiente(Point posicionInicial)
23     {
24         segmentos = new List<Point>();
25         segmentos.Add(posicionInicial);
26         DireccionActual = Direccion.Derecha;
27     }
28
29     4 referencias
30     public List<Point> ObtenerSegmentos() => segmentos;
31
32     3 referencias
33     public Point ObtenerCabeza() => segmentos[0];
34
35     1 referencia
36     public void CambiarDireccion(Direccion nuevaDireccion)
37     {
38         // Evitar dirección opuesta
39         if (nuevaDireccion == Direccion.Arriba && DireccionActual == Direccion.Abajo) return;
40         if (nuevaDireccion == Direccion.Abajo && DireccionActual == Direccion.Arriba) return;
41         if (nuevaDireccion == Direccion.Izquierda && DireccionActual == Direccion.Derecha) return;
42         if (nuevaDireccion == Direccion.Derecha && DireccionActual == Direccion.Izquierda) return;
43         DireccionActual = nuevaDireccion;
44     }
45 }
```



```
38 public void Mover(bool crecer)
39 {
40     Point cabeza = segmentos[0];
41     Point nuevaCabeza;
42
43     switch (DireccionActual)
44     {
45         case Direccion.Arriba: nuevaCabeza = new Point(cabeza.X, cabeza.Y - 1); break;
46         case Direccion.Abajo: nuevaCabeza = new Point(cabeza.X, cabeza.Y + 1); break;
47         case Direccion.Izquierda: nuevaCabeza = new Point(cabeza.X - 1, cabeza.Y); break;
48         default: nuevaCabeza = new Point(cabeza.X + 1, cabeza.Y); break;
49     }
50
51     segmentos.Insert(0, nuevaCabeza);
52     if (!crecer)
53         segmentos.RemoveAt(segmentos.Count - 1);
54 }
55
56 1 referencia
57 public bool ColisionConCuerpo()
58 {
59     Point cabeza = segmentos[0];
60     for (int i = 1; i < segmentos.Count; i++)
61         if (segmentos[i] == cabeza) return true;
62     return false;
63 }
64
65 public void Reiniciar(Point posicionInicial)
66 {
67     segmentos.Clear();
68     segmentos.Add(posicionInicial);
69     DireccionActual = Direccion.Derecha;
70 }
```

```
10 public class Juego
11 {
12     public const int Columnas = 30;
13     public const int Filas = 20;
14     public const int TamanoCelda = 20;
15
16     private Serpiente serpiente;
17     private Alimento alimento;
18
19     4 referencias
20     public int Puntaje { get; private set; }
21     7 referencias
22     public bool EnCurso { get; private set; }
23
24     1 referencia
25     public Serpiente Serpiente => serpiente;
26     1 referencia
27     public Alimento Alimento => alimento;
28
29     1 referencia
30     public Juego()
31     {
32         serpiente = new Serpiente(new Point(5, 5));
33         alimento = new Alimento();
34         Puntaje = 0;
35         EnCurso = true;
36         alimento.GenerarNuevaPosicion(Columnas, Filas, serpiente.ObtenerSegmentos());
37     }
38 }
```



```
34 public void Actualizar()
35 {
36     if (!EnCurso) return;
37
38     bool crecer = serpiente.ObtenerCabeza() == alimento.Posicion;
39
40     // Verificar ANTES de mover si la próxima posición es comida
41     Point cabeza = serpiente.ObtenerCabeza();
42     Point proximaCabeza;
43     switch (serpiente.DireccionActual)
44     {
45         case Direccion.Arriba: proximaCabeza = new Point(cabeza.X, cabeza.Y - 1); break;
46         case Direccion.Abajo: proximaCabeza = new Point(cabeza.X, cabeza.Y + 1); break;
47         case Direccion.Izquierda: proximaCabeza = new Point(cabeza.X - 1, cabeza.Y); break;
48         default: proximaCabeza = new Point(cabeza.X + 1, cabeza.Y); break;
49     }
50
51     crecer = (proximaCabeza == alimento.Posicion);
52     serpiente.Mover(crecer);
53
54     if (crecer)
55     {
56         Puntaje += 10;
57         alimento.GenerarNuevaPosicion(Columnas, Filas, serpiente.ObtenerSegmentos());
58     }
59
60     // Detectar colisión con bordes
61     Point c = serpiente.ObtenerCabeza();
62     if (c.X < 0 || c.X >= Columnas || c.Y < 0 || c.Y >= Filas)
63     {
64         EnCurso = false;
65         return;
66     }
67
68     // Detectar colisión con cuerpo
69     if (serpiente.ColisionConCuerpo())
70         EnCurso = false;
71 }
```

```
73 public void Reiniciar()
74 {
75     serpiente.Reiniciar(new Point(5, 5));
76     Puntaje = 0;
77     EnCurso = true;
78     alimento.GenerarNuevaPosicion(Columnas, Filas, serpiente.ObtenerSegmentos());
79 }
80
81 4 referencias
public void CambiarDireccion(Direccion dir) => serpiente.CambiarDireccion(dir);
```

3. Se diseñaron los controles del formulario:

Control	Nombre	Descripcion
Panel	panelJuego	Tablero de juego (dibujo con GDI+)
Label	lblPuntaje	Muestra el puntaje actual
Timer	timer	Loop del juego (120 ms por tick)

4. Se implemento la lógica de cada clase según la siguiente distribución:



Clase	Responsabilidad	Elementos POO aplicados
Serpiente	Gestiona segmentos, dirección y movimiento	Encapsulamiento, List<Point>, métodos publicos/privados
Alimento	Genera posición aleatoria de la comida	Encapsulamiento, constructor, Random
Juego	Lógica principal: colisiones, puntaje, estado	Composición de objetos, propiedades, métodos
Form1	Interfaz gráfica, eventos de teclado y dibujo	Timer, KeyDown, Paint.
Direccion (enum)	Define las 4 direcciones posibles	Enumeracion, tipo seguro

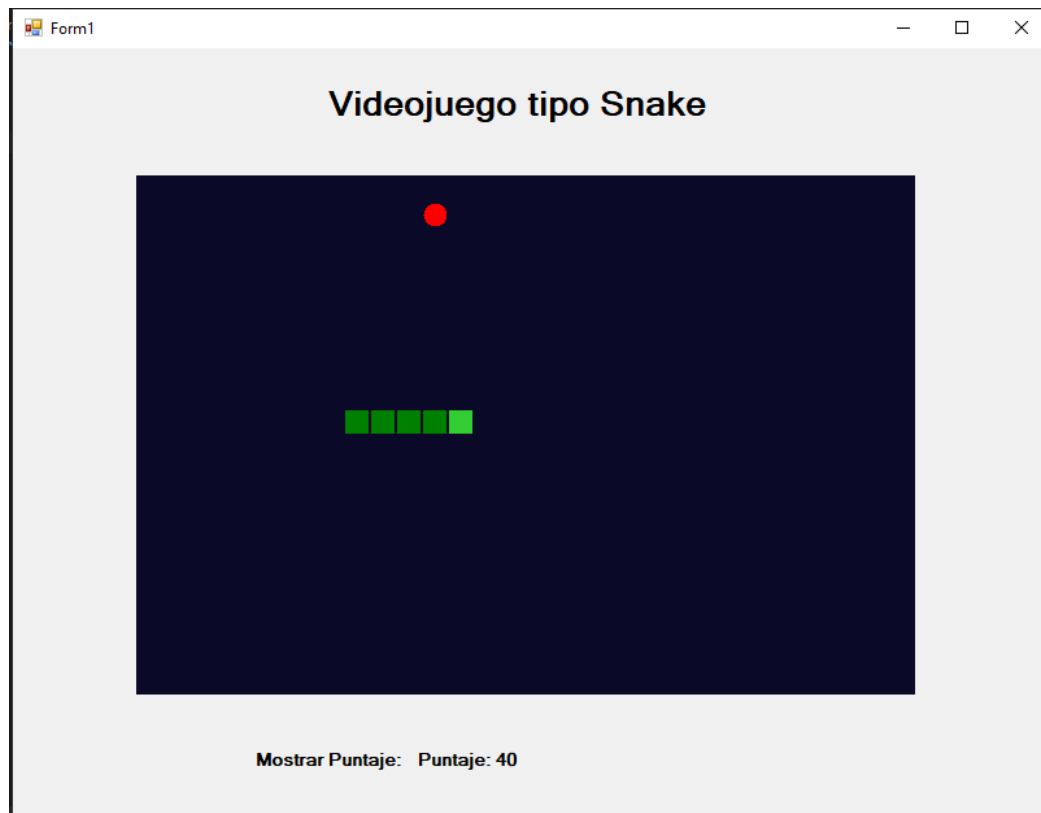
5. En Form1.cs se configuraron los eventos: Timer.Tick para actualizar el juego, Panel.Paint para dibujar con GDI+.

```
42  private void Timer_Tick(object sender, EventArgs e)
43  {
44      juego.Actualizar();
45      lblPuntaje.Text = "Puntaje: " + juego.Puntaje;
46
47      if (!juego.EnCurso)
48      {
49          timer.Stop();
50          lblPuntaje.Text += " | GAME OVER";
51      }
52
53      panelJuego.Invalidate();
54  }
```



```
76 private void PanelJuego_Paint(object sender, PaintEventArgs e)
77 {
78     Graphics g = e.Graphics;
79     int c = Juego.TamanoCelda;
80
81     // Fondo
82     g.Clear(Color.FromArgb(10, 10, 40));
83
84     // Dibujar serpiente
85     var segmentos = juego.Serpiente.ObtenerSegmentos();
86     for (int i = 0; i < segmentos.Count; i++)
87     {
88         Color color = (i == 0) ? Color.LimeGreen : Color.Green;
89         using (SolidBrush brush = new SolidBrush(color))
90             g.FillRectangle(brush, segmentos[i].X * c + 1, segmentos[i].Y * c + 1, c - 2, c - 2);
91     }
92
93     // Dibujar alimento
94     using (SolidBrush brushComida = new SolidBrush(Color.Red))
95     {
96         Point pos = juego.Alimento.Posicion;
97         g.FillEllipse(brushComida, pos.X * c + 1, pos.Y * c + 1, c - 2, c - 2);
98     }
99
100     // Game Over overlay
101     if (!juego.EnCurso)
102     {
103         using (Font font = new Font("Arial", 20, FontStyle.Bold))
104         using (SolidBrush brushTexto = new SolidBrush(Color.White))
105         {
106             g.DrawString("GAME OVER", font, brushTexto, 160, 170);
107             g.DrawString("Presiona R para reiniciar", new Font("Arial", 12), brushTexto, 130, 210);
108         }
109     }
110 }
```

6. Se ejecuto y probó el juego verificando: movimiento de la serpiente, generación aleatoria de alimento, incremento de puntaje, detección de colisiones y pantalla de Game Over.





IV. ANALISIS E INTERPRETACION DE RESULTADOS

¿Que indican los resultados?

El juego funciona correctamente: la serpiente responde al teclado, crece al consumir alimento, el puntaje se incrementa en 10 puntos por cada comida, y el juego detecta colisiones tanto con los bordes del tablero como con el propio cuerpo de la serpiente.

El uso de List<Point> permite que la serpiente crezca de forma dinámica sin límites fijos, lo que demuestra el uso correcto de colecciones en C#.

¿Que se ha encontrado?

Se encontró que el Panel de Windows Forms captura el foco del teclado por defecto, lo que impedía que el formulario recibiera los eventos de las flechas de dirección.

También se identificó que los botones con TabStop activo pueden robar el foco del formulario al ser clickeados, lo que se corrigió estableciendo TabStop = false en todos los controles no interactivos para la lógica del juego.

La separación en 4 clases permitió un código más limpio y fácil de mantener: modificar la velocidad solo requiere cambiar el Interval del Timer, y agregar nuevas funcionalidades (niveles, obstaculos) no afectaría la lógica ya implementada.



V. CUESTIONARIO

1. ¿Por qué se usa List<Point> y no un array para los segmentos de la serpiente?

Porque la serpiente crece dinamicamente al comer. Un array tiene tamaño fijo; con List<Point> se puede insertar al inicio (Insert) y eliminar al final (RemoveAt) de forma sencilla y eficiente.

2. ¿Qué ventaja tiene usar ProcessCmdKey sobre KeyDown para capturar flechas?

ProcessCmdKey se ejecuta antes de que el sistema de foco reparta los eventos a los controles hijos. Esto garantiza que las teclas de dirección siempre lleguen al formulario, sin importar que control tenga el foco en ese momento.

3. ¿Cómo se detecta la colisión con el propio cuerpo?

Después de mover la serpiente, se compara la posición de la cabeza (segmentos[0]) contra todos los demás segmentos de la lista. Si alguno coincide, ColisionConCuerpo() retorna true y el juego termina.

4. ¿Cómo se evita que la serpiente se mueva en dirección opuesta?

En el método CambiarDireccion() de la clase Serpiente se valida que la nueva dirección no sea la opuesta a la actual (Arriba vs Abajo, Izquierda vs Derecha). Si lo es, se ignora el cambio.

5. ¿Qué principio POO representa la clase Juego respecto a Serpiente y Alimento?

Composición. La clase Juego contiene instancias de Serpiente y Alimento como atributos privados, coordinando su interacción sin heredar de ninguna de ellas.

CONCLUSIONES

- Se implementó exitosamente un videojuego Snake funcional en C# Windows Forms aplicando correctamente los principios de POO.
- La separación en clases (Serpiente, Alimento, Juego, Form1) permitió un código organizado y fácil de modificar o ampliar.
- El uso de List<Point> demostró la utilidad de las colecciones dinámicas para modelar estructuras que cambian en tiempo de ejecución.
- Se aprendió que el manejo del foco en Windows Forms es crítico para aplicaciones interactivas; ProcessCmdKey es la forma correcta de capturar teclas de sistema como las flechas.
- El Timer como mecanismo de loop del juego resultó sencillo y efectivo para actualizar el estado y redibujar el tablero de forma periódica.